Applied Deep Learning

Assignment Brief 2026/27

ASSESSMENT WEIGHT	SUBMISSION	ORAL ASSESSMENT
50%	**7pm, Friday 4 December 2026**	**Thursday 10 December 2026**

Overview

This is the specification for the main assignment of the Applied Deep Learning module.

The goal of the project is to perform a systematic investigation of a number of Deep Learning methods in the context of image processing tasks. It gives you an opportunity to put to work many of the concepts you will have been exposed to over the course of the module.

Your implementation should be made in Python running in either Google Colab or in your own choice of local or server-based environment. In all cases Jupyter style notebooks should be supported, as you are required to submit a notebook as a demo of your investigation. Your implementation should be made in Keras / TensorFlow or PyTorch as you see fit, with notebook files such that your code can be opened and run within Google Colab — though it is noted that training may in practice need to be run on a local machine for full training to take place.

Note that you cannot use any alternative data set or coding framework to what is specified in this assignment brief.

Code including the demo notebook should be available in your repository. Models trained can be saved in this same repository. Demo code should be able to use these models as needed.

Assessment

There are three assessment points: one project submission, one short online checkpoint during the semester, and an oral assessment in December.

Component	%	Format
Project — Parts 1, 2 and 3, submitted together	30	Own time, one submission on 4 December
Online checkpoint	5	Individual 5-minute conversation online, Thursday 26 November
Oral assessment	15	Individual 15-minute conversation online, 10 December
Assignment total	50	

Within the project submission, marks are divided as follows: Part 1 — 5%, Part 2 — 14%, Part 3 — 11%.

Your oral assessment mark sets a ceiling on the marks awarded for the project. The rule is set out in full under Oral Assessment below. In short, you may use AI tools freely on the project, but you must be able to explain, modify and justify everything you submit.

Key dates

Date	What is due	Component
Tue 10 Nov 2026	Repository, shared report document and demo.ipynb in place	Part 1
Fri 13 Nov 2026	PREREGISTRATION.md committed to your repository	Part 1
Thu 26 Nov 2026	Online checkpoint — booked 5-minute slots	Checkpoint
Fri 27 Nov 2026	Deadline to request an adjusted oral assessment format	Oral
Fri 4 Dec 2026, 7pm	Report and repository submitted	Parts 1, 2, 3
Thu 10 Dec 2026	Oral assessments — booked 15-minute slots, online	Oral

Task Specification

Part 1: Project Set Up and Pre-registration – 5%

In Part 1 your task is to set up the processes for successfully performing the rest of the assignment, and to commit to a plan before you have any results.

1. Set up a shared "assignment report.docx". The main report should be saved to your official TU Dublin OneDrive folder and shared with me as an examiner with edit access. This document will be your main report and the final version will be submitted at the end of the semester via Brightspace. It is shared so that I can see the work in progress and give you feedback on it during the semester.
2. Set up a private GitHub repository just for this assignment and share it with me. All code lives here.
3. Create a demo.ipynb file and use the file to train a basic MNIST digits model (see week 1). Configure your development environment of choice with access to your repository and commit demo.ipynb by 10 November 2026.
4. Commit a one-page PREREGISTRATION.md by 13 November 2026, as set out below.

Pre-registration

Three of the five marks for Part 1 are for the pre-registration. It must state:

> (i) the four comparisons you have chosen to investigate in Part 2, one of which must be a dataset variation;
> (ii) for each one, what you expect to happen, in which direction, and roughly by how much — a number or a range, not "it should improve";
> (iii) the metric you will judge each comparison by, and the rule you will use to decide whether a difference you observe is real rather than run-to-run noise;
> (iv) one sentence on what result would surprise you.

You are not marked on being right. The marks are for the quality of the reasoning behind your predictions, and later, in the report, for your account of where prediction and observation diverged and why. A pre-registration vague enough that nothing could contradict it scores poorly. A confident prediction that turns out to be wrong, well explained afterwards, scores highly.

The file is timestamped when you commit it and should not be rewritten afterwards. If your plan changes, which is normal, add a dated entry to the bottom of the file explaining what changed and why.

Part 2: Age Estimation Investigation – 14%

The core of this project is based around a simple task — building and evaluating a model for age estimation based on the "facial age" dataset on Kaggle:

https://www.kaggle.com/datasets/frabbisw/facial-age

Note that there are a number of facial age datasets on Kaggle, so please make sure you are using the link above to get the correct one. The dataset has a simple structure consisting of numerous directories corresponding to individual ages.

Data preparation

- Save out 10% of the data to be used as a test set. This test set should not be used for validation but instead be held out for end testing. Remember that this should be randomly selected.
- Make a training/validation split from the remainder of the dataset. The specific split here is up to you, but a validation set of roughly 10–20% of the total data probably makes sense for a dataset this size.
- Folder names give the ages for each group, but you will want to create a CSV file that actually contains the ages explicitly. While you are creating this, you should also create additional explicit labels to class the ages into broad groups: e.g. child 0–12, teen 13–19, youth 20–30, mid 31–45, mature 46–60, older 61+. The specific categories you use are up to you, but try to use a smaller, well balanced set of categories to improve performance on the classification task.
- **Examine the dataset before you split it.** Consider whether the images are independent of one another, and whether a purely random split is therefore the right thing to do here. Document what you found and what you decided.

Baselines

Establish and report trivial baselines before reporting any model:

> — for regression: a predictor that always outputs the training-set mean age;
> — for classification: both a majority-class predictor and a stratified random predictor.

Every subsequent result must be reported against these. A model that does not beat its baseline is a legitimate finding and is marked as such, provided you identify it and say so. A report that does not establish baselines cannot score well, because the results in it cannot be interpreted.

The investigation

This part of the task is to perform a number of analyses based on training from scratch to predict age both as a real-valued number and as an age category. Your models should be designed to minimise overfitting as appropriate, and you should consider the investigation of hyperparameters, though a full grid search will not be feasible. This initial set of models should be based on hand-crafted networks only — pre-trained

networks for this task exist and backbone networks could improve performance a lot, but the goal here is for you to explore network variations for yourself.

Carry out the four comparisons you named in your pre-registration, chosen from the menu below.

CNN kernel variations: different activation functions; different kernel sizes.

Network variations: different network depths; with and without pooling; with and without skip connections; different numbers of fully connected layers.

Dataset variations (one of your four must come from here): 1 channel vs 3 channel; data augmentation to enhance the dataset.

A variation of your own: come up with your own variation in model design or architecture for investigation. A wide amount of variation is possible here.

For each of your four comparisons:

> — run each condition with at least three random seeds, and report the spread as well as the central value;
> — state explicitly whether the difference you observe is larger than the seed-to-seed variation within a single condition, and therefore whether you are entitled to claim an effect;
> — hold everything else fixed, and state in the report what "everything else" was.

Model variations both for regression (specific age) and age category should be produced. For age category, investigate the different types of output layer activation function that are possible. In all cases record your results as graphs for training and validation data and report the test result after training has been completed. Select whatever loss functions or metrics you think are appropriate based on notes provided in class, or more widely based on what is appropriate in a regression or classification task.

Null results. Your report must contain a short section listing the comparisons in which you found no effect you can defend, and saying why you think that is. Four clean improvements from four comparisons will be read as a question about experimental control rather than as a strong result, and you should expect to be asked about it.

Model saving

From the various models above, save the two top-performing models. Your demo code should be able to load these saved models and demonstrate performance on the held-out dataset. Your test dataset should be saved and made available for your demo code to use via a Google Drive download or similar.

Part 3: Pre-training, Transfer and Backbones – 11%

In Part 3 we switch from a direct focus on the age prediction task and instead look at the production of a pre-trained model based on an autoencoder, which you will then apply to the age categorisation task.

For this part of the task, take the non-test data and split it into two subsets, Block 1 and Block 2. One of these subsets will be used to train the autoencoder and the other will be used to build an age classification model. Your first task is to make these subsets and save them to disk for use later. Make sure to randomly select the images.

As with Part 2, the tasks below do not build on each other and can in some ways be considered independent.

Autoencoder modelling (Block 1). Building on your best performing model to this point, construct an autoencoder model to attempt to reproduce input images. Discuss the appropriate use of loss functions and metrics for evaluation. Save the model for the demo notebook and use it later.

Transfer learning (Block 2). Using the model saved above, investigate the use of transfer learning to build a network for Block 2 images for the age classification task. Compare the result to your original models from Part 2.

Backbone model re-use. Using the Block 1 data, fine-tune a generic image processing model to perform the age category classification task. The generic model should, for example, be pre-trained on ImageNet rather than on a specific age classification task. Compare this backbone-based model to your own homegrown models.

The comparisons are what is being marked here, not the models. Both comparisons above must be made against the baselines established in Part 2 and with the same treatment of run-to-run variation. A backbone model outperforming your hand-built model by a wide margin is expected; the marks are for whether you can say why, whether the comparison was fair, and what it cost — in parameters, in training time, and in the ability to explain what the model is doing.

Online Checkpoint – 5%

On Thursday 26 November 2026 you will have a five-minute individual checkpoint online. Bring one result you have obtained so far — a training curve, a comparison, a baseline, anything you have actually run — share your screen, and talk me through it.

You will be asked what the result shows, what you expected, and what you intend to do next. There is nothing to prepare beyond having done some of the work, and no marks depend on the result being good or the project being on schedule. Missing the checkpoint without a documented reason scores zero for this component.

Oral Assessment – 15%

On Thursday 10 December 2026, each student has an individual 15-minute online conversation with me about their own submitted work. You will have your own repository and report open in front of you, share your screen when requested, and may refer to your work freely throughout.

Structure

The conversation follows the structure below.

1. Orientation (1 min) — walk me through your repository and tell me where things are.
2. Accounting for a result (4 min) — I pick one comparison from your report and ask you to explain the result and how confident you are that it is real.
3. Counterfactual (3 min) — what would happen if we changed one thing, and why.
4. Live micro-modification (3 min) — where in your code would a specified change go, and what else would have to change with it. You are not asked to write working code under time pressure.
5. Diagnosis (2 min) — I show you an output from one of your own runs and ask you to interpret it.
6. Limits (2 min) — what is your model bad at, and what in your evaluation could have hidden that from you.

Marks are for the substance of what you say, not the fluency of your delivery. Hesitation, thinking out loud, correcting yourself, and saying "I don't know, but here is how I would find out" all score perfectly well. There is no criterion for confidence or presentation.

The oral cap

Your band in the oral assessment sets a ceiling on the marks awarded for the project. It does not affect the marks for the checkpoint or for the oral itself.

Band	Descriptor	Cap	What this looks like
4	Full command of the submitted work	No cap	Explains design choices and why alternatives were rejected; interprets own results unprompted; answers counterfactuals about the work
3	Sound understanding, some gaps	70%	Explains what was done and why; hesitant on parts of the pipeline; recovers with prompting
2	Partial understanding	55%	Describes the work accurately but cannot account for specific choices or specific numbers; cannot answer counterfactuals
1	Familiar with the work only as a reader	40%	Can summarise the report but cannot locate anything in the code or explain any number in it
0	Cannot account for the submitted work	0%	Referred to the unfair practice process

Arrangements

- Students must book their own oral-assessment slot using the scheduling link that will be made available closer to the submission deadline. Orals are recorded so that a second marker can moderate and so that there is a record if you wish to query your mark.
- If you have approved accommodations, or if there is anything about this format that would disadvantage you, contact me by 27 November 2026 and an adjusted format will be arranged.
- Attendance is required. Not attending without a documented and approved reason means the project cannot be awarded marks.

Submission Specification

Report

The report should be 8–12 pages in length, using standard margins, an 11pt readable font (Arial or equivalent), 1.15 spacing between lines, and approximately 4pt spacing between paragraphs. Appendices sit outside this limit.

The first page is a standalone cover page and should include your name, your student number, your declaration of AI use, and the following statement:

> "I confirm that the document and related code here is all my own work, that I can explain and defend all of it, and that I did not engage in unfair practice in any way."

Do not waste space on a table of contents.

This report should be a well written, suitably structured and appropriately referenced document that justifies decisions, provides results as well as your analysis. The report should not give a background explanation of Deep Learning or related models, and should not simply describe the models that were created. Instead, write it as a narrative of your investigation, clearly showing the major steps taken, any dead ends and any learnings. The following sections should minimally be included:

> — Part 1, including what you pre-registered and where prediction and outcome diverged;
> — Part 2, including baselines, your four comparisons, and the null results section;
> — Part 3;
> — AI collaboration critique (see below);
> — Appendices, which do not count towards the page limit.

Code

Code is submitted via the shared GitHub repository. A requirements.txt should be generated and maintained to list the packages needed to run your code. Some flexibility on style will be given so long as the marking scheme is adhered to.

Your code submission should consist of two elements. First, each component above should have its own training notebook, clearly demonstrating the data processing and training of your models; saving and testing can also be explored in these notebooks. Second, one extra notebook should be included as a demo notebook. The demo notebook should not contain training code, but should contain code to load your best models from the web, load your test data from the web, and demo the models at work. Submitting a non-working version of this demo notebook will impact marking.

When submitting notebooks, make sure they are purged of data. Otherwise notebook files can be REALLY BIG and be a problem for GitHub and version control. Submitting notebooks with embedded data may lead to marks being lost.

Conditions of submission

The following are conditions of submission rather than marked criteria. Work that does not meet them will not be marked:

> — a private GitHub repository shared with me, containing all code, the demo notebook and PREREGISTRATION.md;
> — a report shared with me on OneDrive with edit access, and the final version submitted via Brightspace;
> — a working demo notebook that loads your saved models and held-out test set from the web and runs without modification;
> — attendance at your scheduled oral assessment.

Use of AI Tools

You may use AI tools freely for the project. You are not required to log your prompts and no marks depend on doing so. You are required to declare, in one short paragraph on your cover page, what you used and for what — for example, “used an LLM to generate a first version of the training loop, to debug a tensor shape error, and to redraft section 4”.

Everything you submit is yours to defend. You may not submit code or text that you could not explain, modify or justify if asked. The oral assessment is where this is checked and its outcome takes precedence over the submitted artefacts. AI detection tools are not used and their output is not treated as evidence.

AI collaboration critique

> Include a section of approximately half a page in your report describing two specific occasions on which an AI tool gave you a suggestion that was wrong, subtly incorrect, or worse than the alternative. For each, give the suggestion, how you established that it was wrong, and what you did instead.
> Generic reflection on the limitations of AI scores nothing; specific, technical, verifiable instances score well. If you used these tools throughout and never caught one in an error, report that instead and say what it suggests.

Unfair Practice

Copying text or code from other sources without appropriate acknowledgement is unfair practice, including self-plagiarism, and is subject to a grade of 0 on the relevant parts of the assignment and, or, full disciplinary procedures. The checkpoint and the oral assessment are individual, closed assessments, and assistance of any kind in those is unfair practice. If you are in doubt about anything here, ask me.

For reference, see the Students' Union guide to unfair practice.

Marking

Parts 1, 2 and 3 are marked against the descriptors below, applied to each part separately and then subject to the oral cap. The checkpoint and the oral are marked against their own published rubrics.

Band	Descriptor
70–100%	The work is correct and the analysis is insightful. Experimental design is sound: baselines are present, variation is quantified, claims are proportionate to the evidence, and the limits of what can be concluded are stated without being asked. The report reads as an account of an investigation, including where it went wrong.
60–69%	The work is correct and competently analysed. Baselines and variation are handled properly. Analysis explains what happened but is thinner on why, and some claims run slightly ahead of the evidence.
50–59%	The work is largely complete and the results are reported accurately, but the analysis mostly describes rather than explains. Baselines may be present but under-used. Differences are claimed without much attention to whether they are real.
40–49%	The required elements are present but the treatment is mechanical. Results are reported without baselines or without acknowledgement of run-to-run variation, and the discussion does not go beyond restating the numbers.
0–39%	Elements are missing, results are not reported in a usable form, or the analysis is absent or incorrect.

Submission Details

The project is due at 7pm on Friday 4 December 2026. Extensions will not be granted except in the case of documented and approved personal circumstances. The following penalties apply to late submissions:

Late submission penalties

Submission delay	Penalty
Less than 1 day late	7%
Less than 2 days late	14%
Less than 3 days late	21%
Less than 4 days late	28%
Less than 5 days late	35%
Less than 6 days late	42%
Less than 1 week late	49%
1 week late or more	Submissions not accepted

A late submission may require the oral assessment to be rescheduled.

All students should be available for their online oral assessment on Thursday 10 December 2026. Students must book their own individual 15-minute slot using the scheduling link made available closer to the submission deadline; slots will run during the class period.